PROMPT 3 — Servidor REST del Backend

> Documento-prompt para una IA implementadora. Debe **crear el código PHP** del servidor REST y entregarlo funcionando + pruebas automáticas en verde. No dejar "a medias": responde a las rutas y al contrato de la sección 3 exactamente.
>
> **Ante cualquier duda o falta de información: PÁRATE y pregunta al responsable del proyecto ANTES de escribir o modificar nada.** No asumas ni inventes.

---

1. Rol y tarea

Eres un ingeniero backend PHP (sin frameworks, PHP plano).

Tarea: implementar el **servidor REST** que expone las **2 rutas de la sección 3.1**, delegando en `LogicaNegocio` (Prompt 2). La BBDD NO se toca desde aquí.

2. Contexto del proyecto

- La placa BLE emite un iBeacon; la app Android capta el beacon y hace **POST** con el valor.
- El teléfono envía `{"minor": 1234}` (+ metadatos opcionales `uuid`, `major`, `tx_power`, `nombre_emisora`) y espera `200 {"ok": true, "id": N}`.
- La web pide por **GET** todas las mediciones y las pinta.
- `LogicaNegocio` ya existe (Prompt 2): `guardarMedicion(Medicion)` y `recuperarMedicion()`.

3. Diseño a implementar

3.1 Rutas REST

Verbo	Ruta	Delegación	Respuesta
`POST`	`/mediciones`	`ServidorREST::guardarMedicion()`	`200 {"ok": true, "id": N}`
`GET`	`/mediciones`	`ServidorREST::recuperarMedicion()`	`200 {"mediciones": [Medicion...]}`
cualquier otra	—	`rutaNoEncontrada()`	`404 {"error": "ruta no encontrada"}`

3.2 Contrato JSON

POST (petición mínima del teléfono):

```
{ "minor": 1234 }
```

(o con metadatos: {"minor":1234,"uuid":"EPSG-GTI-PROY-3A","major":2816,"tx_power":4,"nombre_emisora":"Minerva_ELBACON"})

POST (respuesta 200):

```
{ "ok": true, "id": 1 }
```

GET (respuesta 200):

```
{ "mediciones": [ { "id": 1, "uuid": "EPSG-GTI-PROY-3A", "major": 2816,
                    "minor": 1234, "tx_power": 4,
                    "nombre_emisora": "Minerva_ELBACON",
                    "fecha_hora": "2026-09-28 09:00:00" } ] }
```

3.3 Clase NUEVA ServidorREST + front controller

```
      ------------------- ServidorREST (NEW) -------------------
      |
      | // (new) crea su LogicaNegocio interna cuando lo necesita
      |
      |
cuerpo: Text --> guardarMedicion() -->      // (new) POST /mediciones
      |
      |
      --> recuperarMedicion() -->           // (new) GET /mediciones
      |
      |
      --> rutaNoEncontrada() -->            // (new) 404
      |
      ------------------------------------------------------------

      ------------------- index.php (NEW) front controller -------------
      |
      | 1. cabecera base: Content-Type: application/json; charset=utf-8
      | 2. crea ServidorREST()
      | 3. despacha segun REQUEST_METHOD + URI
      |    POST /mediciones  -> guardarMedicion( file_get_contents("php://input")
)
      |    GET  /mediciones  -> recuperarMedicion()
      |    otro              -> rutaNoEncontrada()
      |
      ------------------------------------------------------------------
```

3.4 Ficha de métodos

Método	Firma	Comportamiento
`guardarMedicion()`	`cuerpo: Text`	`json_decode` → si `null` → `400 {"error":"cuerpo JSON invalido"}`. Si falta `minor` → `400 {"error":"falta el campo 'minor'"}`. Llama a `LogicaNegocio.guardarMedicion()`. `InvalidArgumentException` → `400` con su mensaje; `PDOException` → `500 {"error":"error interno del servidor"}`. Éxito → `200 {"ok": true, "id": <id>}`.
`recuperarMedicion()`	`--> Text` (emite JSON)	Llama a `LogicaNegocio.recuperarMedicion()` → `200 {"mediciones":[...]}`. `PDOException` → `500`.

Cabecera base (en `index.php`): `Content-Type: application/json; charset=utf-8`.

4. Ficheros a entregar

En `backend/`:

- `public/index.php` — front controller (3.3).
- `rest/ServidorREST.php` — clase con los 3 métodos de la ficha.
- `.htaccess` — reescritura de URLs hacia `index.php` (si Apache).

> La ruta canónica es `/mediciones`. No añadas archivos en la ruta: es REST limpio.

5. Requisitos de código y de comentarios (OBLIGATORIOS)

1. Comentarios **en español**, simples y estructurados.
   - Cabecera por fichero y por método: `PROPOSITO` / `PARAM.` / `RETORNA`.
2. Fidelidad al estilo del proyecto: separadores `// ----`, cierre `// ()`.
3. Nunca SQL aquí; solo delegar en `LogicaNegocio`.

6. Pruebas automáticas (OBLIGATORIAS)

6.1 `backend/tests/ServidorRESTTest.php` (PHPUnit)

Con `LogicaNegocio` real y BBDD de prueba `mediciones_db_test` (misma técnica que el Prompt 2: `BD_NOMBRE` sobreescrita en el bootstrap; captura salida con `ob_start()/ob_get_clean()`):

1. `guardarMedicion('{"minor": 1234}')` → código `200` y JSON con `"ok": true` y `"id"` numérico.
2. `guardarMedicion('no-json')` → `400`.
3. `guardarMedicion('{}')` → `400` y el cuerpo nombra `minor`.
4. `guardarMedicion('{"minor":99999}')` → `400` (fuera de rango SMALLINT → validación).
5. `recuperarMedicion()` tras guardar → `200` y JSON con una fila `minor 1234`.
6. `recuperarMedicion()` sin datos → `200 {"mediciones":[]}`.

6.2 Integración HTTP real (script `backend/tests/api_test.sh` o manual)

Sirve el proyecto con **Apache (XAMPP)**: copia el contenido de `backend/public` dentro de `C:\xampp\htdocs\` (preferiblemente como raíz, o en una subcarpeta) y abre `http://localhost/<carpeta>/mediciones`, donde `<carpeta>` es el nombre que le pongas en `htdocs` (vacío si va en la raíz):

```
curl -i -X POST http://localhost/<carpeta>/mediciones \
     -H "Content-Type: application/json" -d '{"minor":1234}'        # -> 200
{"ok":true,"id":N}
curl -i http://localhost/<carpeta>/mediciones                        # -> 200
{"mediciones":[...]}
curl -i http://localhost/<carpeta>/otracosa                          # -> 404
```

6.3 Ejecución

```
vendor/bin/phpunit --configuration backend/phpunit.xml backend/tests
```

7. Verificación y entrega

- [ ] `POST /mediciones` → `200 {"ok": true, "id": N}`; `GET /mediciones` → lista JSON de Mediciones.
- [ ] Errores `400/404/500` con cuerpo JSON de error.
- [ ] El servidor nunca toca la BBDD directamente.
- [ ] Comentarios en español. Tests en verde. Resumen final: ficheros, comandos, resultado.